HOTEL BOOKING API

Project Requirements • System Flows • Entity Relationship Diagram

Project type: Multi-hotel booking platform
Document: Project planning and database design
Version: 1.0 — Planning draft
Backend stack: ASP.NET Core Web API (.NET 10), C#, EF Core 10, PostgreSQL

Purpose

This document combines the initial Software Requirements Specification (SRS), user roles, business rules, customer and administrator workflows, API endpoint plan, and the supplied database ERD image. It is the planning baseline to review before implementation.

Project direction: Build a realistic multi-hotel platform where customers can discover hotels, search rooms, make reservations, and manage bookings, while administrators manage hotels, rooms, and reservations.

1. Project Overview

Hotel Booking API is a RESTful backend system for a multi-hotel booking platform. It supports hotel discovery, room search, reservation management, simulated payments, and hotel operations such as check-in and check-out.

1.1 Objectives

• Allow customers to find hotels and available rooms.

• Support online room reservations and booking history.

• Prevent overlapping reservations for the same physical room.

• Calculate and preserve the price agreed at booking time.

• Record payment attempts and their outcomes.

• Provide secure authentication and role-based access.

• Provide administration tools for hotel and room management.

1.2 Technology Stack

Technology	Purpose
C# / ASP.NET Core Web API (.NET 10)	Backend application and REST endpoints
PostgreSQL	Relational database
Entity Framework Core 10	ORM, data access, and migrations
ASP.NET Core Identity + JWT	User accounts, password hashing, authentication
Swagger / OpenAPI	API documentation and manual testing
Docker / Docker Compose	Containerized development and deployment
Postman / xUnit	API testing and automated tests
Git / GitHub	Version control and collaboration

2. User Roles and Permissions

Role	Main permissions
Guest (public)	Browse/search hotels; view hotel and room information; register and log in.
Customer	Manage own profile; search availability; create bookings; view own bookings/payments; make simulated payments; cancel eligible bookings.
Admin	Manage hotels, room types, and rooms; view all bookings; manage booking status; perform check-in/check-out; view payment records.
Hotel Manager (future)	Manage assigned hotels and their rooms/bookings only. Not required for initial Version 1 unless approved.

Security rule: Public registration must never allow a user to select or assign themselves the Admin role. Customers must not be able to read or modify another customer's private bookings or payments.

3. Functional Requirements

Requirement	Description
FR-01 Authentication	Register, log in, issue and validate JWT access tokens, retrieve profile, and enforce role-based access.
FR-02 Hotel management	Create, list, view, update, deactivate/delete hotels; search by name, city, and location.
FR-03 Room management	Create and manage room types and physical rooms; set capacity and price; activate/deactivate rooms.
FR-04 Room availability	Search by hotel/city, check-in/check-out dates, and guest count; return only rooms available for the entire date range.
FR-05 Booking management	Create bookings, calculate nights and total price, view booking details/history, cancel eligible bookings, and manage statuses.
FR-06 Payment management	Create payment records, record amount/method/status, link payments to bookings, and simulate success/failure.
FR-07 Hotel operations	Perform guest check-in/check-out and view upcoming/current reservations.

4. Non-Functional Requirements

Area	Requirement
Security	Hash passwords; protect private endpoints with JWT; enforce roles and ownership; validate input; do not store raw card details or expose secrets.
Performance	Use async database calls, pagination, useful indexes, and avoid unnecessary queries.
Reliability	Use transactions for critical operations, prevent double booking, and keep booking/payment states consistent.
Maintainability	Separate controllers, services, interfaces, DTOs, models, and data access; use DI, migrations, and automated tests.
Observability	Use structured logs and consistent error responses; add monitoring in deployment.

5. Business Rules

Rule	Description
BR-01 Dates	Check-in must be before check-out; check-in cannot be in the past; minimum stay is one night.
BR-02 Capacity	Number of guests cannot exceed the room's allowed capacity.
BR-03 Availability	A room cannot have overlapping bookings that block availability. Cancelled, rejected, and expired bookings do not block availability.
BR-04 Price	Total = price per night × number of nights. Store the agreed nightly price and total as a booking snapshot.
BR-05 Payment	A failed payment is never treated as paid. Confirmation follows the configured payment policy.
BR-06 Cancellation	Customers may cancel only their own eligible bookings; cancellation policy determines eligibility and refund behavior.
BR-07 Authorization	Only authorized admins manage hotel inventory and all reservations; customers access only their own private records.
BR-08 Concurrency	Availability must be checked safely during booking creation; concurrent requests must not reserve the same room for overlapping dates.

6. System Flows

6.1 Customer booking flow

Browse hotels → search by city and dates → view available rooms → select a room → authenticate if needed → enter guest details → submit booking → backend validates dates and availability → calculate price → create pending booking → payment → confirm booking if payment succeeds.

6.2 Administrator flow

Admin logs in → API validates JWT → checks Admin role → opens administration functions → manages hotels, rooms, bookings, or check-in/check-out → saves changes to PostgreSQL.

6.3 Booking and payment flow

Validate dates and guest count → check availability → calculate total → create pending booking → create payment attempt → process simulated payment → if successful, mark payment Paid and booking Confirmed; if unsuccessful, mark payment Failed and keep or expire the pending booking according to the configured timeout policy.

Critical design note

A simple availability check in a controller is not enough. Two customers may attempt to reserve the same room simultaneously. Booking creation must use a database transaction and an appropriate concurrency-safe strategy. The implementation will also define how long pending bookings hold inventory.

7. Booking and Payment Statuses

Booking status	Meaning
Pending	Waiting for payment or confirmation.
Confirmed	Reservation is confirmed.
CheckedIn	Guest has arrived.
CheckedOut	Guest has departed.
Cancelled	Reservation was cancelled.
Expired	Pending reservation passed its confirmation/payment deadline.

Payment status	Meaning
Pending	Waiting for payment result.
Paid	Payment succeeded.
Failed	Payment attempt failed.
Refunded	Payment was fully refunded.
PartiallyRefunded	Part of the payment was refunded.

Booking status and payment status are separate. A booking can have multiple payment attempts, so the payment relationship is one booking to many payment records.

8. Initial API Endpoint Plan

Module	Method	Endpoint	Access
Auth	POST	/api/auth/register	Public
Auth	POST	/api/auth/login	Public
Hotels	GET	/api/hotels	Public
Hotels	GET	/api/hotels/{id}	Public
Hotels	POST	/api/hotels	Admin
Hotels	PUT	/api/hotels/{id}	Admin
Hotels	DELETE	/api/hotels/{id}	Admin
Rooms	GET	/api/hotels/{hotelId}/rooms	Public
Rooms	POST	/api/rooms	Admin
Rooms	PUT	/api/rooms/{id}	Admin
Rooms	GET	/api/rooms/available	Public
Bookings	POST	/api/bookings	Customer
Bookings	GET	/api/bookings/my	Customer
Bookings	GET	/api/bookings/{id}	Owner/Admin
Bookings	PATCH	/api/bookings/{id}/cancel	Owner/Admin
Payments	POST	/api/payments	Customer
Payments	GET	/api/payments/my	Customer
Admin	GET	/api/admin/bookings	Admin
Admin	PATCH	/api/admin/bookings/{id}/check-in	Admin
Admin	PATCH	/api/admin/bookings/{id}/check-out	Admin

Endpoint list is an initial contract. Request/response DTOs, filtering parameters, pagination, and exact status codes will be specified during implementation planning.

9. Database Design — ERD

The following diagram is the ERD image supplied for this project. It includes Hotel, RoomType, Room, User, Role, UserRole, Booking, and Payment.

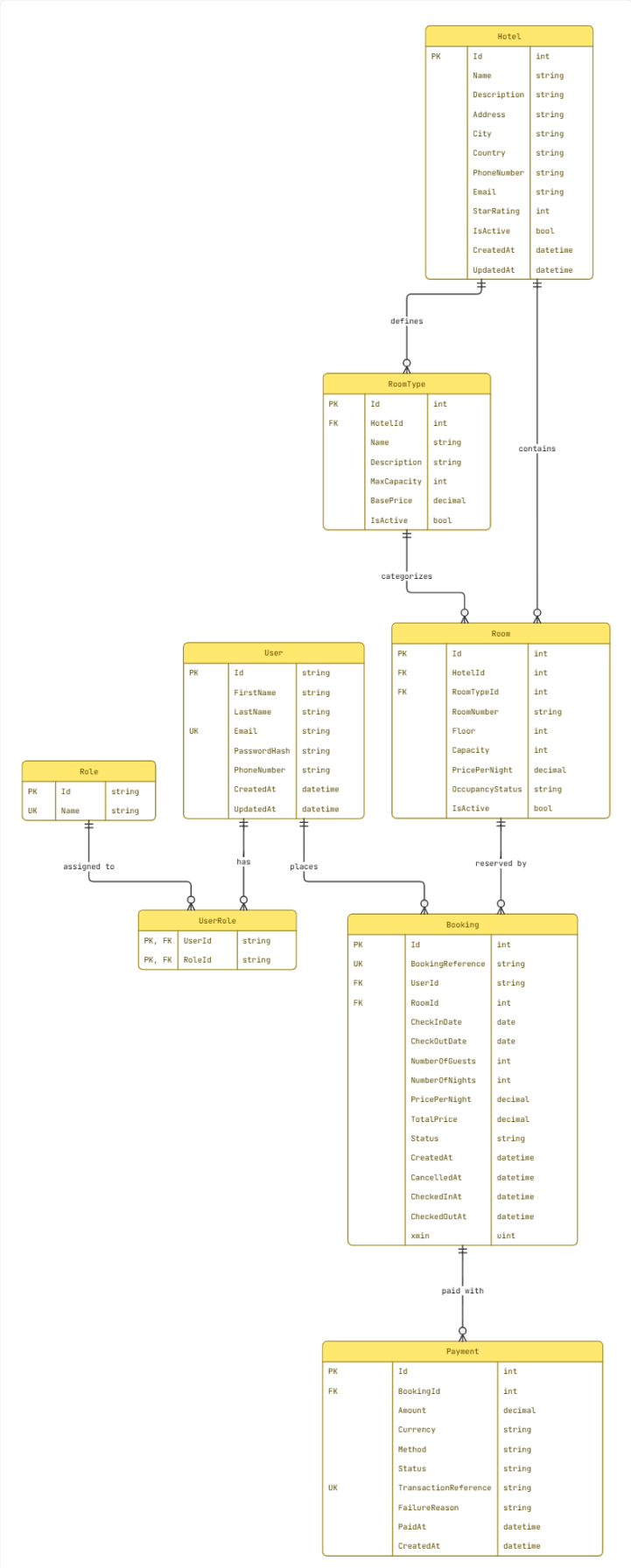


Figure 1. Supplied Hotel Booking API Entity Relationship Diagram.

10. ERD Review and Recommended Adjustments

The supplied ERD is a strong starting point. Before implementing migrations, review these details so the database supports the business rules correctly.

Area	Recommendation
User / Role	If using ASP.NET Core Identity, use IdentityUser/ApplicationUser and the Identity role and user-role tables. Avoid creating a second custom User/Role model that duplicates Identity.
Hotel / RoomType / Room	The diagram shows HotelId on both RoomType and Room. If each Room belongs to a RoomType and each RoomType belongs to one Hotel, RoomTypeId already identifies the hotel. Keeping both can create inconsistent data unless the relationship is deliberately constrained. Prefer deriving Hotel through RoomType, or enforce that both hotel references match.
Room capacity	Decide whether Capacity belongs to RoomType or Room. If all rooms of a type share capacity, store it on RoomType; if individual rooms can differ, store it on Room.
Room pricing	Choose one source of truth for the standard price. The diagram has BasePrice on RoomType and PricePerNight on Room. Keep both only if Room.PricePerNight is an override or current effective price.
Room occupancy	OccupancyStatus should describe current physical occupancy (e.g., Vacant/Occupied/OutOfService), not date-range availability. Availability must be computed from bookings.
Booking	Keep PricePerNight and TotalPrice as booking snapshots. Add currency, cancellation reason, and optional expiry timestamp if required by policy.
Payment	TransactionReference should be unique when provided. Consider provider name, failure reason, refund fields, and a provider idempotency key for real payment integration.
Dates and time	Use date-only values for hotel check-in/check-out dates, and timestamps for events such as CreatedAt, PaidAt, CheckedInAt, and CheckedOutAt.
Indexes and constraints	Add indexes for hotel city, room type, room number, booking user/date/status, and payment booking/status. Add unique constraints for hotel-room number and booking reference.
Deletion	Prefer IsActive/soft deletion for hotels and rooms with booking history; do not hard-delete records referenced by completed bookings.

Important ERD relationship

A room can have many bookings over time, but overlapping active bookings must be prevented. A relational foreign key alone cannot enforce this business rule. PostgreSQL can enforce it with an exclusion constraint on a date range, or the application can use a carefully designed transaction and locking strategy.

11. Project Scope and Phases

Phase	Deliverables
Version 1 — Core	Authentication/authorization; hotel, room type, and room CRUD; search and availability; booking and cancellation; simulated payments; admin booking management; check-in/check-out.
Version 2 — Advanced	Real payment gateway; hotel manager role; reviews and ratings; image uploads; email notifications; discounts/coupons; refund processing; analytics and reports.

12. Acceptance Criteria

- A customer can register and log in securely.
- An administrator can create and manage hotels, room types, and rooms.
- A customer can search available rooms for a valid date range and guest count.
- A customer can book an available room and receive a correct price calculation.

• The system prevents overlapping bookings for the same room.

• Customers can view and manage only their own bookings and payment history.

• Payment attempts are recorded with accurate statuses.

• Administrators can manage reservations and check-in/check-out.

• The API has automated tests for important business rules and runs in Docker.

13. Implementation Roadmap

Step	Work
1	Approve SRS, roles, booking policy, payment policy, and ERD.
2	Create ASP.NET Core project, configure PostgreSQL, EF Core, Swagger, and Docker.
3	Implement Hotel, RoomType, and Room models, relationships, migrations, and CRUD.
4	Implement DTOs, validation, services, DI, and consistent error handling.
5	Implement Identity, JWT, roles, and ownership authorization.
6	Implement room search and concurrency-safe availability/booking logic.
7	Implement cancellation, simulated payments, and hotel operations.
8	Add pagination, filtering, logging, indexes, and security hardening.
9	Write unit/integration tests, document API, containerize, and deploy.

14. Open Decisions Before Coding

Decision	Recommended initial choice
Payment	Simulated payment flow; no real card data or gateway in Version 1.
Roles	Customer + Admin initially; Hotel Manager can be added later.
Booking model	One physical room per booking in Version 1.
Pending reservation hold	Define a short expiry window and release inventory when it expires.
Cancellation/refund	Define cutoff time and refund rules before implementing cancellation.
Deployment	Docker locally first; AWS deployment after tests and security review.

Document status: Planning draft — review and approve the open decisions before creating the project or database migrations.